

TTS Skill 能力总结与从零复刻指南

这份文档用于向他人解释一套本地 TTS / 声音克隆 / 长文本配音能力，也可以直接交给 Codex，让它在另一台 Apple Silicon Mac 上复刻同类能力。

它复刻的是“能力系统”，不是某个人的声音、频道风格或固定路径。新用户只需要准备自己的参考音频和逐字参考文本，就可以生成自己的克隆声音，并把它封装成 Codex 可调用的 tts skill。

1. 这套 tts skill 是什么

这套 tts skill 是一个本地配音能力层。它把文本、Markdown 文稿或长篇口播稿，转成稳定的 WAV 音频，并支持声音克隆、长文本分段、断点修复、局部重生和 Codex 自动调用。

底层技术路线：

▼ Plain Text 复制代码

```
1  Codex / 脚本 / 自动化 / 内容工厂 / 桌面宠物
2      |
3      v
4  Codex tts skill
5      |
6      +--> generate_voiceover.py          # 短文本 / 一般脚本
7      |
8      +--> generate_comfort_voiceover.py # 长文 / 播客 / YouTube 口播
9          |
10         v
11         tts_daemon_client.py
12         |
13         v
14         tts_daemon.py
15         |
16         v
17         mlx-speech + fish-s2-pro
18         |
19         v
20         本地 WAV 输出
```

Codex / 脚本 / 自动化 / 内容工厂 / 桌面宠物

|

v

Codex tts skill

|

4415字



```
+--> generate_voiceover.py # 短文本 / 一般脚本
```

```
|
```

```
+--> generate_comfort_voiceover.py # 长文 / 播客 / YouTube 口播
```

```
|
```

```
v
```

```
tts_daemon_client.py
```

```
|
```

```
v
```

```
tts_daemon.py
```

```
|
```

```
v
```

```
mlx-speech + fish-s2-pro
```

```
|
```

```
v
```

```
本地 WAV 输出
```

核心特点：

- 本地运行，不依赖云端 TTS API。
- 使用 mlx-speech 和 fish-s2-pro。
- 支持普通 TTS，也支持声音克隆。
- 支持中文口播、播客、YouTube 旁白、课程讲解、通知语音、桌面宠物音效。
- 支持把长文本拆成自然语义短句，逐段生成后再合并。
- 支持常驻 daemon 预热模型，减少每次重新加载模型的等待。
- 支持短参考音频优先，降低克隆加载慢和不稳定的问题。
- 支持局部重生，长文中某一段有问题时，不需要整篇重跑。
- 明确禁止 macOS say 兜底，避免混入不同声音。

2. 已经实现的能力

2.1 普通文本转语音

输入一段文本，输出一个 WAV 文件。

适合：

- 短通知
- 小宠物语音
- 单句口播
- 产品提示音



- 测试模型是否可用

2.2 自定义声音克隆

用户提供：

- reference_audio
- reference_text

系统用这段参考音频克隆用户自己的声音。参考文本必须和参考音频逐字对应。

推荐参考音频：

- 10 到 20 秒
- 单人声
- 无背景音乐
- 无明显混响
- 自然口播
- WAV、mono、24kHz、16-bit PCM

2.3 文稿转播客 / YouTube 口播

输入 .txt 或 .md 文稿，Codex 可以先理解文稿，再转成更适合 TTS 的输入：

- 补标点
- 合并断裂字幕
- 删除 Markdown 噪音
- 调整为自然口播节奏
- 在重点处加轻量标签，如 [calm]、[serious]、[emphasis]
- 用短句和段落停顿塑造节奏

注意：不要把规则说明文字、Markdown 标题层级、列表符号原样读出来，除非用户明确要求逐字朗读。

2.4 长文本不断点生成

长文不能一次性丢给模型。正确做法是：

1. 先把整篇文稿转成口播稿。
2. 按语义拆成短片段。
3. 每段单独生成 WAV。
4. 检查每段是否异常截断。
5. 修剪每段头尾静音。

4415字 6. 按语义插入停顿。



7. 合并成完整音频。
8. 输出时间表和分段表，方便复查和局部重生。

当前经验：

- 推荐每段 1 个想法。
- 目标单段口播字数不超过 20 个中文字符。
- 硬上限约 22 个中文字符，除非明确要更快更密。
- 长句、枚举、转折句要拆短。
- 不要用 75 字、90 字这种大段去赌模型稳定性。

2.5 断点和截断风险处理

常见问题是：模型生成到一半突然停，例如一句诗或一句观点只读到前半句。

已沉淀的处理规则：

- 单段过长时拆短。
- 避免长枚举一次生成。
- 避免孤立引号、奇怪符号影响模型。
- 避免一个短句里堆多个情绪标签。
- 如果生成结果触发可疑条件，先不合并。
- 可疑条件包括：
 - 单段时长异常长；
 - 生成帧数接近硬上限；
 - 文本超过硬字数；
 - `generated_tokens == max_new_tokens`；
 - 片段明显只读了一半。

正确策略不是“用别的声音补上”，而是拆短后用同一个克隆声重试。

2.6 局部重生

长文生成后，如果第 403 段有问题，不需要从头生成。

可以从某个 segment id 开始重生：

```
python "$HOME/.codex/skills/tts/scripts/generate_comfort_voiceover.py" \  
--text-file /path/to/script.md \  
--out "$HOME/Codex_Workspace/08_OUTPUTS/audio/episode_listen.wav" \  
--work-dir "$HOME/Codex_Workspace/08_OUTPUTS/audio/episode_work" \  
--start-id 403 \  
--qc-window 21:00,25,check_21m00_21m25.wav
```

4415字



这对长篇播客非常重要，因为它避免“一处断句错误导致整集重跑”。

2.7 daemon 预热加速

直接每次调用 `mlx-speech` 会反复加载模型和参考音频，速度慢。

更好的方式是：

- 启动 `tts_daemon.py`
- 预加载 `fish-s2-pro`
- 缓存默认参考音频编码
- 后续通过 `tts_daemon_client.py` 发请求

这样适合：

- 批量生成固定通知语音
- 长文多段生成
- 桌面宠物语音包
- 内容工厂批量旁白

2.8 固定语音库

对于桌面宠物、任务完成提醒、定时提醒，不推荐每次任务结束都实时调用大模型或 TTS。

更合理的是：

- 先用克隆声音批量生成固定 WAV；
- 按场景分类保存；
- 运行时只用 `afplay` 播放；
- 不实时生成；
- 不阻塞电脑。

适合分类：

- 任务完成
- 需要用户输入
- 早晨开工
- 中午吃饭
- 下午续航
- 晚上收工
- 深夜休息
- 点击撒娇
- 双击大笑



- 拖动求饶
- 长按摸头

3. 可以应用到哪些场景

内容生产

- YouTube 口播旁白
- 播客配音
- 读书分享音频
- 知识课程讲解
- 小红书 / 视频号短视频配音
- 多版本标题、开场、结尾 CTA 试听

AI 内容工厂

- 文稿生成后自动转语音
- 分章节生成音频
- 对某个有问题片段局部重生
- 输出 timing.tsv 给字幕、视频剪辑、Remotion 使用
- 作为视频管线里的 voiceover stage

桌面伙伴和通知系统

- Codex 任务完成固定语音提醒
- 需要用户输入提醒
- 饭点、喝水、休息提醒
- 宠物交互语音包
- 不同时间段随机播放不同语音

自动化系统

- 工作日报语音播报
- 自动化脚本完成提示
- 本地 Agent 状态提示
- 长任务完成后播放固定奖励语音



产品原型

- App 内语音提示
- 本地虚拟助手
- 交互式学习工具
- 角色语音包
- 情绪化反馈音效

4. 最重要的避坑经验

1. 不要使用 macOS say 兜底。
它会让同一成品里混入另一种声音，破坏一致性。
2. 不要在克隆失败后自动 fallback 到 base TTS。
应该同声重试一次；仍失败就停下来报告。
3. 不要把长文一次性丢给模型。
长文必须语义分段，逐段生成。
4. 不要用太长参考音频。
默认优先 10–20 秒短参考，长参考只做备份。
5. 不要让单段太长。
单段越长，越容易截断、读漏、拖慢、生成失败。
6. 不要让 `generated_tokens == max_new_tokens` 的片段直接进入成品。
这是截断风险信号。
7. 不要在 TTS 文本里塞太多标签。
[calm]、[serious]、[emphasis] 够用。一个片段最多一个标签。
8. 不要使用未知标签，例如 [pause]。
停顿靠标点、分段和合并阶段的静音间隔。
9. 不要把引号、Markdown 符号、列表编号原样塞给模型。
先清理成自然口播文本。
10. 不要把宠物实时互动和长文 TTS 混成一个运行时系统。
宠物运行时应播放预生成音频；长文 TTS 应离线生成。
11. 不要复制 .venv 到新电脑。
新机器应重新用 uv 创建 Python 3.13 虚拟环境。
12. Hugging Face / Xet 下载不稳定时，设置：

```
export HF_HUB_DISABLE_XET=1
```

5. 从零复刻指南



5.1 环境要求

- Apple Silicon Mac
- macOS
- uv
- ffmpeg
- Python 3.13
- Codex 本地可读写 \$HOME/.codex/skills

检查:

```
uname -m
which uv
which ffmpeg
python3 --version
```

如果缺少 ffmpeg:

```
brew install ffmpeg
```

如果缺少 Python 3.13:

```
uv python install 3.13
```

5.2 推荐路径

不要写死用户名。

```
export CODEX_WORKSPACE_ROOT="${CODEX_WORKSPACE_ROOT:-$HOME/Codex_Workspace}"
export LOCAL_TTS_DIR="$CODEX_WORKSPACE_ROOT/04_AGENT_SYSTEMS/local_tts"
export TTS_OUTPUT_DIR="$CODEX_WORKSPACE_ROOT/08_OUTPUTS/audio"
export TTS_SKILL_DIR="$HOME/.codex/skills/tts"
```

目录结构:

```
$CODEX_WORKSPACE_ROOT/
04_AGENT_SYSTEMS/
local_tts/
README.md
refs/
README_REF_AUDIO.md
default_ref_12s.wav
default_ref_12s.txt
default_ref.wav
default_ref.txt
scripts/
```




```
tts_cli.py
tts_daemon.py
tts_daemon_client.py
tests/
demo_script.txt
test_basic_tts.sh
test_clone_tts.sh
08_OUTPUTS/
audio/

$HOME/
.codex/
skills/
tts/
SKILL.md
scripts/
generate_voiceover.py
generate_comfort_voiceover.py
```

5.3 安装

```
mkdir -p "$LOCAL_TTS_DIR/refs"
mkdir -p "$LOCAL_TTS_DIR/scripts"
mkdir -p "$LOCAL_TTS_DIR/tests"
mkdir -p "$TTS_OUTPUT_DIR"
mkdir -p "$TTS_SKILL_DIR/scripts"

cd "$LOCAL_TTS_DIR"
uv python install 3.13
uv venv --python 3.13 .venv
source .venv/bin/activate
uv pip install --upgrade pip setuptools wheel
uv pip install mlx-speech
```

验证:

```
mlx-speech --help
mlx-speech tts --list-models
```

确认 fish-s2-pro 可用。



放入：

```
$LOCAL_TTS_DIR/refs/default_ref_12s.wav
```

```
$LOCAL_TTS_DIR/refs/default_ref_12s.txt
```

可选备份：

```
$LOCAL_TTS_DIR/refs/default_ref.wav
```

```
$LOCAL_TTS_DIR/refs/default_ref.txt
```

转换音频：

```
ffmpeg -i input.m4a \
```

```
-ac 1 \
```

```
-ar 24000 \
```

```
-sample_fmt s16 \
```

```
-t 12 \
```

```
"$LOCAL_TTS_DIR/refs/default_ref_12s.wav"
```

把对应逐字文本写入：

```
nano "$LOCAL_TTS_DIR/refs/default_ref_12s.txt"
```

5.5 必须创建的脚本

最小可用版本需要：

- scripts/tts_cli.py
- scripts/tts_daemon.py
- scripts/tts_daemon_client.py
- \$HOME/.codex/skills/tts/SKILL.md
- \$HOME/.codex/skills/tts/scripts/generate_voiceover.py
- \$HOME/.codex/skills/tts/scripts/generate_comfort_voiceover.py

如果是让 Codex 实现，建议不要手写每个脚本，而是把本文末尾的“给 Codex 的快速启动提示词”直接复制给它，让它创建、测试和修复。

6. 快速使用命令

短文本：

```
python "$HOME/.codex/skills/tts/scripts/generate_voiceover.py" \
```

```
--text "今天我们来测试一段本地声音克隆配音。" \
```

```
--out "$HOME/Codex_Workspace/08_OUTPUTS/audio/demo.wav"
```

长文：



```
python "$HOME/.codex/skills/tts/scripts/generate_comfort_voiceover.py" \
--text-file "$HOME/Codex_Workspace/demo_script.md" \
--out "$HOME/Codex_Workspace/08_OUTPUTS/audio/demo_long_listen.wav" \
--work-dir "$HOME/Codex_Workspace/08_OUTPUTS/audio/demo_long_work" \
--qc-window 0:00,120,first_2m.wav
```

只准备分段，不生成：

```
python "$HOME/.codex/skills/tts/scripts/generate_comfort_voiceover.py" \
--text-file "$HOME/Codex_Workspace/demo_script.md" \
--out "$HOME/Codex_Workspace/08_OUTPUTS/audio/demo_long_listen.wav" \
--work-dir "$HOME/Codex_Workspace/08_OUTPUTS/audio/demo_long_work" \
--prepare-only
```

健康检查：

```
"$HOME/Codex_Workspace/04_AGENT_SYSTEMS/local_tts/.venv/bin/python" \
"$HOME/Codex_Workspace/04_AGENT_SYSTEMS/local_tts/scripts/tts_daemon_client.p
y" \
--health
```

停止 daemon：

```
"$HOME/Codex_Workspace/04_AGENT_SYSTEMS/local_tts/.venv/bin/python" \
"$HOME/Codex_Workspace/04_AGENT_SYSTEMS/local_tts/scripts/tts_daemon_client.p
y" \
--stop
```

7. 给 Codex 的快速启动提示词

把下面整段复制给新电脑上的 Codex：

你现在要在这台 Mac 上搭建一套通用的本地 TTS / 声音克隆 / Codex skill 能力。

目标：

1. 本地安装 mlx-speech。
2. 使用 Fish Audio S2 Pro，也就是 fish-s2-pro 模型。
3. 支持普通文本转语音。
4. 支持声音克隆：reference_audio + reference_text。
5. 支持长文口播、播客、YouTube 旁白。
6. 支持长文本自然语义分段，不断点，不混声。
7. 支持常驻 daemon，缓存模型和默认参考声，提升批量生成速度。
8. 输出 WAV 到用户工作区的 08_OUTPUTS/audio。
9. 创建可复用 CLI：scripts/tts_cli.py。



10. 创建 daemon: scripts/tts_daemon.py。
11. 创建 daemon client: scripts/tts_daemon_client.py。
12. 创建 Codex skill: \$HOME/.codex/skills/tts。
13. 创建长文生成 wrapper: generate_comfort_voiceover.py。

硬约束：

- 不要写死 /Users/具体用户名。
- 所有路径基于 Path.home() 和环境变量 CODEX_WORKSPACE_ROOT。
- 默认 WORKSPACE_ROOT 为 \$HOME/Codex_Workspace。
- 修改已有文件前先备份。
- 不要复制旧机器的 .venv，新机器必须重建虚拟环境。
- 不要使用 macOS say 作为 fallback。
- 不要在 clone 失败后 fallback 到 base TTS。
- 不要混合不同声音片段。
- 不要把长稿一次性丢给模型。
- 默认优先使用 10-20 秒短参考音频。
- 不要使用超过 20 秒的参考音频作为默认参考。
- 不要使用未知标签，例如 [pause]。
- 不要堆叠情绪标签，一个片段最多一个。
- 如果某段 generated_tokens == max_new_tokens，视为可能截断，必须拆短该段或提高 token 后重生。
- 如果某段两次同一克隆声重试仍失败，停止并报告，不要生成混声成品。
- 如果 Hugging Face / Xet 下载失败，设置 HF_HUB_DISABLE_XET=1 后重试。

路径：

```
WORKSPACE_ROOT="${CODEX_WORKSPACE_ROOT:-$HOME/Codex_Workspace}"
PROJECT_DIR="$WORKSPACE_ROOT/04_AGENT_SYSTEMS/local_tts"
OUTPUT_DIR="$WORKSPACE_ROOT/08_OUTPUTS/audio"
SKILL_DIR="$HOME/.codex/skills/tts"
```

请执行：

1. 创建目录：
 - \$PROJECT_DIR/refs
 - \$PROJECT_DIR/scripts
 - \$PROJECT_DIR/tests
 - \$OUTPUT_DIR
 - \$SKILL_DIR/scripts

2. 检查环境：

- uname -m，确认 Apple Silicon 为 arm64。



- which uv。
- which ffmpeg。
- python3 --version。
- 如果没有 ffmpeg, 用 Homebrew 安装。
- 如果没有 Python 3.13, 用 uv python install 3.13。

3. 安装:

```
cd "$PROJECT_DIR"
uv venv --python 3.13 .venv
source .venv/bin/activate
uv pip install --upgrade pip setuptools wheel
uv pip install mlx-speech
```

4. 验证:

```
mlx-speech --help
mlx-speech tts --list-models
```

确认 fish-s2-pro 可用。

5. 创建 refs/README_REF_AUDIO.md, 说明用户需要放入:

- refs/default_ref_12s.wav
- refs/default_ref_12s.txt

可选:

- refs/default_ref.wav
- refs/default_ref.txt

6. 实现 scripts/tts_cli.py:

- 支持 --text。
- 支持 --text-file。
- 支持 --out。
- 默认模型 fish-s2-pro。
- 支持 --voice auto/base/clone。
- 支持 --ref-audio。
- 支持 --ref-text。
- 支持 --ref-text-file。
- 无 reference audio 时普通 TTS。
- 有 reference audio 和 ref text 时声音克隆。
- 输出目录不存在时自动创建。
- 用 subprocess 调用 mlx-speech CLI。
- 设置 HF_HUB_DISABLE_XET=1。



7. 实现 `scripts/tts_daemon.py`:

- 启动本地 HTTP 服务, 默认 `127.0.0.1:8765`。
- 加载 `fish-s2-pro`。
- 编码并缓存默认 `reference audio`。
- 接收 `text/out/voice/ref-audio/ref-text`。
- 输出 `wav`。
- 返回 `voice_mode`、`duration`、错误信息。

8. 实现 `scripts/tts_daemon_client.py`:

- 支持 `--text / --text-file / --out`。
- 支持 `--voice auto/base/clone`。
- 支持 `--health`。
- 支持 `--stop`。
- `daemon` 不存在时自动启动。
- 不打印敏感环境变量。

9. 创建 Codex skill:

- `$SKILL_DIR/SKILL.md`
- `$SKILL_DIR/scripts/generate_voiceover.py`
- `$SKILL_DIR/scripts/generate_comfort_voiceover.py`

10. `generate_voiceover.py` 要支持:

- 短文本和普通脚本。
- 先优化 TTS 输入。
- 按语义分段。
- 调用 `daemon client`。
- 使用同一个 `voice route`。
- 输出 `parts` 目录。
- 合并 `wav`。
- 失败时同声重试一次。
- 不 fallback 到 `say` 或 `base TTS`。

11. `generate_comfort_voiceover.py` 要支持:

- 长文播客和 YouTube 旁白。
- `--prepare-only`。
- `--start-id` 局部重生。
- `--qc-window` 导出抽查片段。
- 输出 `script_segments.tsv`。
- 输出 `timing.tsv`。

4415字 - 每段目标 ≤ 20 中文字符, 硬限制 ≤ 22 。



- 根据标点和段落意义插入停顿。
- 修剪每段头尾近静音。
- 检测可疑截断片段。
- 发现高风险片段时不要合并成最终成品。

12. 创建测试：

- tests/demo_script.txt。
- tests/test_basic_tts.sh。
- tests/test_clone_tts.sh。

13. 最终验证：

- 普通 TTS 能生成 wav。
- 如果用户已提供 default_ref_12s.wav 和 default_ref_12s.txt, 克隆 TTS 能生成 wav。
- 长文 prepare-only 能生成 script_segments.tsv。
- 长文生成能输出 final wav、raw merged wav、parts、timing.tsv、qc sample。
- daemon health 正常。

14. 输出总结：

- 安装是否成功。
- Python 版本。
- mlx-speech 是否可用。
- fish-s2-pro 是否可用。
- 普通 TTS 输出路径。
- 克隆 TTS 输出路径, 如果参考音频存在。
- 长文测试输出路径。
- daemon 是否可用。
- 如果失败, 失败在哪一步, 下一步修复建议。

8. 给别人讲解时的推荐话术

可以这样解释：

这不是一个简单的“文本转语音脚本”，而是一套本地声音生产系统。

第一层是 mlx-speech + fish-s2-pro，解决本地生成声音。

第二层是声音克隆，用户把自己的 10 到 20 秒干净口播和逐字稿放进去，系统就能用同一声音生成新内容。

第三层是长文本工程化。长文不能直接丢给模型，否则会断点、截断、漏读。所以我们把文章先变成口播稿，再按语义切成短句，逐句生成，检查风险，最后合并。



第四层是 Codex skill。这样用户不用每次记命令，只要告诉 Codex“用 tts skill 把这篇稿子生成播客”，Codex 就能按标准流程执行。

真正的关键不是“能不能出声”，而是：

- 同一个声音保持一致；
- 长文不断点；
- 失败不混声；
- 局部能重生；
- 能接入内容工厂和自动化。

9. 最小验收标准

复刻完成后，至少要通过：

- `mlx-speech --help` 正常。
- `mlx-speech tts --list-models` 能看到或调用 `fish-s2-pro`。
- 普通 TTS 生成 WAV。
- 克隆 TTS 生成 WAV。
- `daemon health` 正常。
- 长文 `--prepare-only` 生成 `script_segments.tsv`。
- 长文完整生成后有：
 - final listening WAV
 - raw merged WAV
 - parts directory
 - `script_segments.tsv`
 - `timing.tsv`
 - QC sample
- 没有使用 macOS say。
- 没有混用 base voice 和 cloned voice。
- 高风险截断片段没有被直接合并进成品。